

Object Oriented Programming

Java
Threads



What is a Thread?

- Definition
 - Single sequential flow of control within a program
 - Facility to allow multiple activities to **coexist** within a single process
 - Represents a separate path of execution of a group of statements
 - Example:
 - Operating System
 - Handling Several Clients
- Why threads?
 - Need to handle concurrent processes



Process and Threads

- A process is a thread in execution.
- A process may be divided into number of **independent units** known as **threads**.
- Threads are **light-weight** processes within a process .
- A process is a collection of one or more threads and associated system resources.



Difference between Process and Threads

Process	Thread
Process can be divided into multiple threads	Threads cannot be sub divided.
Each process has its own memory space	Threads of the same process share a common memory space
It is difficult to create a process	It is easy to create a thread.

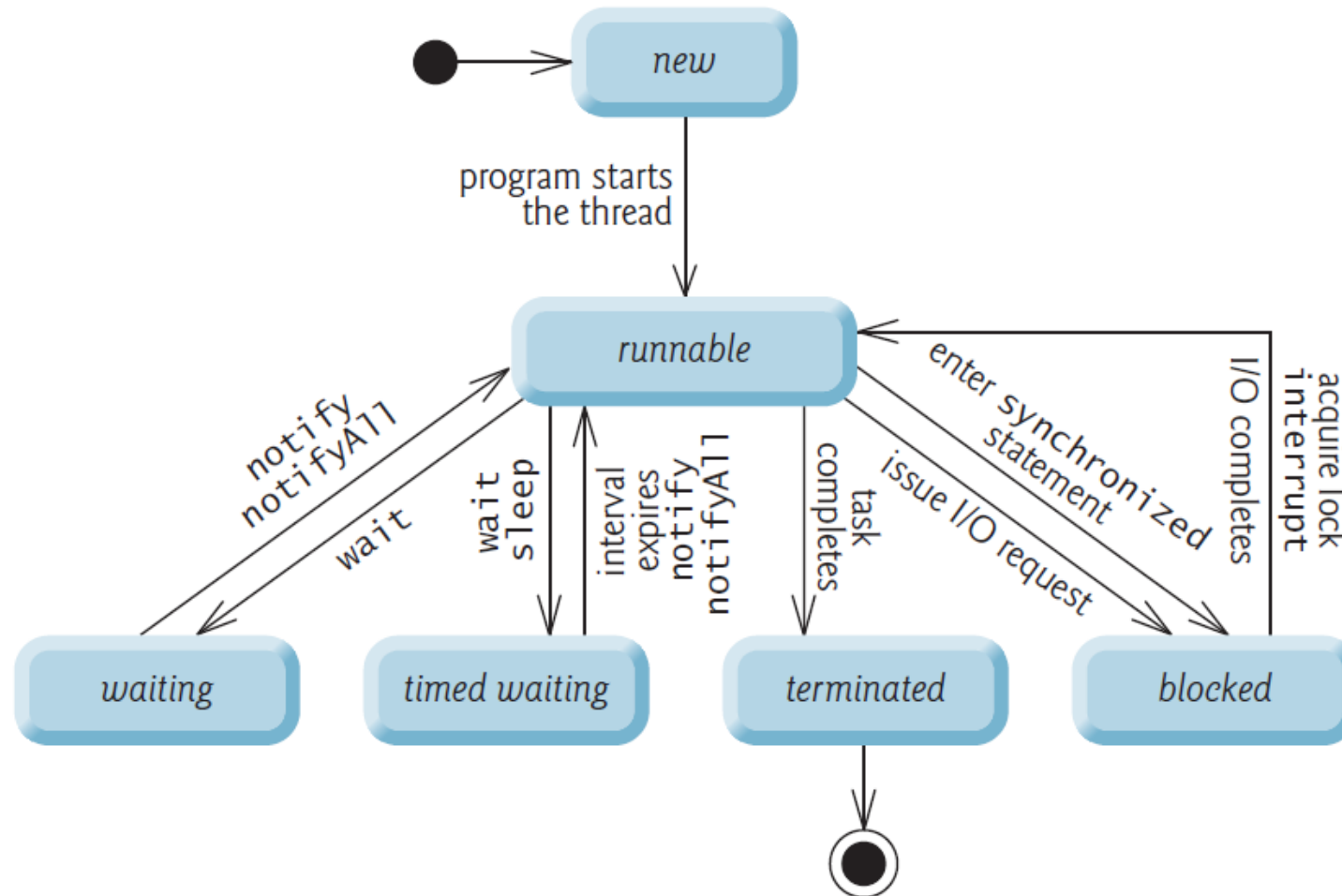


Thread States

- A thread can be in one of several possible states:
 1. **New**
 - ☐ Can run but not yet given the chance
 2. **Runnable**
 - ☐ considered to be executing its task
 - ☐ In control of CPU
 3. **Waiting**
 - ☐ Waits for another thread to perform a task
 4. **Timed waiting**
 - ☐ Waits for a specified interval of time
 5. **Blocked**
 - ☐ Waiting for some resource or event to occur
 6. **Terminated**
 - ☐ It successfully completes its task or otherwise terminates (perhaps due to an error).



Thread States



Thread Priorities

- Definition:
 - Integer value ranging from 1 to 10
 - Higher the thread priority → larger chance of being executed first
 - Example:
 - Two threads are ready to run
 - First thread: priority of 5, already running
 - Second thread = priority of 10, comes in while first thread is running
- Why priorities?
 - Determine which thread **receives CPU control** and gets to be executed first



Thread Priorities

- **Context switch**
 - Occurs when a thread **snatches** the control of CPU from another
 - When does it occur?
 - Running thread voluntarily relinquishes CPU control
 - Running thread is preempted by a higher priority thread
- More than one highest priority thread that is ready to run
 - Deciding which receives CPU control depends on the operating system
 - Windows 95/98/NT: Uses **time-sliced round-robin**
 - Solaris: Executing thread should voluntarily relinquish CPU control



Thread Class

- **Constructor**

1. **Thread()**

- Creates a new Thread object.

2. **Thread(String name)**

- Creates a new Thread object with the specified *name*.

3. **Thread(Runnable target)**

- Creates a new Thread object based on a *Runnable* object. Target refers to the object whose run method is called.

4. **Thread(Runnable target, String name)**

- Creates a new Thread object with the specified *name* and based on a *Runnable* object.



Thread Class

- Thread Constants

1. public final static int MAX_PRIORITY
 - The maximum priority value, 10.
2. public final static int MIN_PRIORITY
 - The minimum priority value, 1.
3. public final static int NORM_PRIORITY
 - The default priority value, 5.



Thread Class

- Thread Methods

- public static Thread `currentThread()`
 - Returns a reference to the thread that is currently running.
- public final String `getName()`
 - Returns the name of this thread.
- public final void `setName(String name)`
 - Renames the thread to the specified argument *name*. May throw *securityException*.
- public final int `getPriority()`
 - Returns the priority assigned to this thread.
- public final boolean `isAlive()`
 - Indicates whether this thread is running or not.



Creating Java Threads

- **Two Ways** of Creating and Starting a Thread
 1. Extending the *Thread* class
 2. Implementing the *Runnable* interface

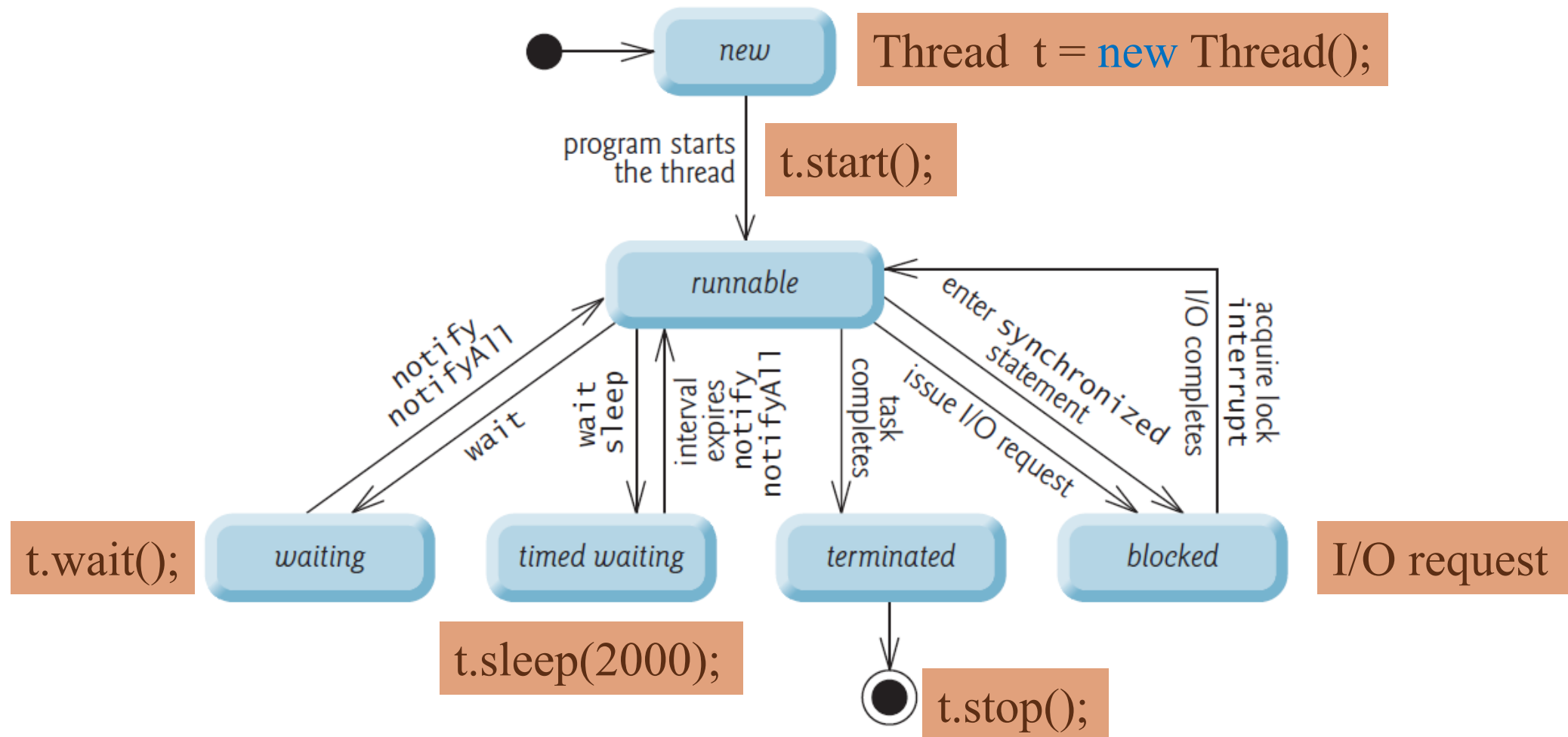


Extending Thread Class

- The subclass **extends** *Thread* class
 - The subclass overrides the **run()** method of *Thread* class
- An object instance of the subclass can then be created
- Calling the **start()** method of the object instance starts the execution of the thread
 - Java runtime starts the execution of the thread by calling *run()* method of object instance



Thread States



Two Schemes of starting a thread

In two ways a thread can be started from a subclass

1. The *start()* method is not in the constructor of the subclass
 - The start() method needs to be explicitly invoked after object instance of the subclass is created in order to start the thread
2. The *start()* method is in the constructor of the subclass
 - Creating an object instance of the subclass will start the thread



start() method is Not in the constructor

```
public class PrintNameThread extends Thread {  
    PrintNameThread(String name) {  
        super(name);  
    }  
    public void run() {  
        String name = getName();  
        for (int i = 0; i < 100; i++) {  
            System.out.print(name);  
        }  
    }  
}
```



start() method is Not in the constructor

```
public class ExtendThreadClassTest1 {  
    public static void main(String args[]) {  
        PrintNameThread pnt1 = new PrintNameThread("A");  
        pnt1.start(); // Start the first thread  
        PrintNameThread pnt2 = new PrintNameThread("B");  
        pnt2.start(); // Start the second thread  
    }  
}
```



start() method is in a constructor

```
public class PrintNameThread extends Thread {  
    PrintNameThread(String name) {  
        super(name);  
        start(); //runs the thread once instantiated  
    }  
    public void run() {  
        String name = getName();  
        for (int i = 0; i < 100; i++) {  
            System.out.print(name);  
        }  
    }  
}
```



start() method is in a constructor

```
public class ExtendThreadClassTest2 {  
    public static void main(String args[]) {  
        PrintNameThread pnt1 = new PrintNameThread("A");  
        PrintNameThread pnt2 = new PrintNameThread("B");  
    }  
}
```



Implementing Runnable Interface

- The Runnable interface should be **implemented** by any class whose instances are intended to be executed as a thread
- The class must define ***run()*** method of no arguments
 - The *run()* method is like *main()* for the new thread
- Provides the means for a class to be active while not extending Thread
 - A class that implements Runnable can run without extending Thread by instantiating a Thread instance and passing itself in as the target



Two Schemes of starting a thread

- Two Ways of Starting a Thread For a class that implements Runnable
 1. Caller thread creates Thread object and starts it explicitly after an object instance of the class that implements Runnable interface is created
 - The *start()* method of the Thread object needs to be explicitly invoked after object instance is created
 2. The Thread object is created and started within the constructor method of the class that implements Runnable interface
 - The caller thread just needs to create object instances of the Runnable class



Caller thread creates a Thread object and starts it explicitly

```
public class PrintNameRunnable implements Runnable {  
    String name;  
    PrintNameRunnable(String name) {  
        this.name = name;  
    }  
    // Implementation of the run() defined in the Runnable interface.  
    public void run() {  
        for (int i = 0; i < 10; i++) {  
            System.out.print(name);  
        }  
    }  
}
```



Caller thread creates a Thread object and starts it explicitly

```
public class RunnableThreadTest1 {  
    public static void main(String args[]) {  
        PrintNameRunnable pnt1 = new PrintNameRunnable("A");  
        Thread t1 = new Thread(pnt1);  
        t1.start();  
    }  
}
```



Thread object is created and started within a constructor

```
public class PrintNameRunnable implements Runnable {  
    Thread thread;  
    PrintNameRunnable(String name) {  
        thread = new Thread(this, name);  
        thread.start();  
    }  
    // Implementation of the run() defined in the Runnable interface.  
    public void run() {  
        String name = thread.getName();  
        for (int i = 0; i < 10; i++) {  
            System.out.print(name);  
        }  
    }  
}
```



Thread object is created and started within a constructor

```
public class RunnableThreadTest2 {  
    public static void main(String args[]) {  
        /** Since the constructor of the PrintNameRunnable object creates a Thread  
        object and starts it, there is no need to do it here. */  
        new PrintNameRunnable("A");  
        new PrintNameRunnable("B");  
        new PrintNameRunnable("C");  
    }  
}
```



Extending Thread vs. Implementing Runnable Interface

- Choosing between these two is a matter of taste
- Implementing the Runnable interface
 - May take more work since we still
 - Declare a Thread object
 - Call the Thread methods on this object
 - Your class **can still extend** other class
- Extending the Thread class
 - Easier to implement
 - Your class can **no longer extend** any other class



Synchronization

- **Race conditions** occur when multiple, asynchronously executing threads access the same object (called a **shared** resource) returning unexpected (wrong) results
- Example:
 - Threads often need to share a common resource i.e. a file, with one thread reading from the file while another thread writes to the file
- They can be avoided by synchronizing the threads which access the shared resource



Synchronization: Locking an Object

- A thread is synchronized by becoming an **owner** of the object's monitor
 - Consider it as locking an object
- A thread becomes the owner of the object's monitor in one of two ways
 - Option 1: Use *synchronized* method
 - Option 2: Use *synchronized* statement on a common object
- *synchronized* keyword involves locking and unlocking before entering into **synchronized method or block** thread needs to acquire the **lock**, at this point it reads data from main memory than cache and when it releases the lock, it flushes write operation into main memory which eliminates memory inconsistency errors.

